

Chapter 3

Research Methodology

3.1 Introduction

In this chapter, we outline the methodological framework. Specifically, we define the research design, data collection strategy, preprocessing procedures, feature engineering process, graph construction approach, proposed model architecture, training and validation strategy, evaluation metrics, implementation tools, and key methodological assumptions. Figure 3.1 presents the primary methodological flow of the proposed framework.

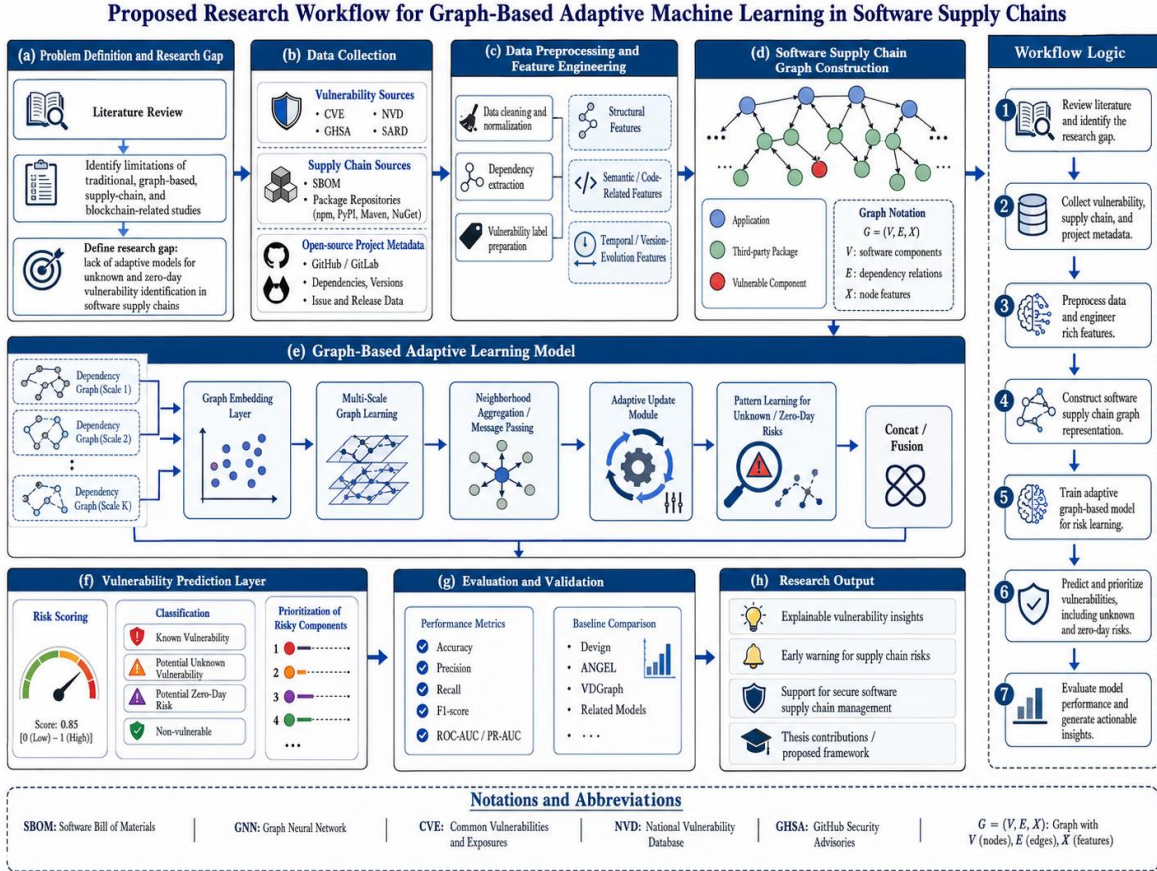


Fig. 3.1. Research workflow of the proposed graph-based adaptive machine learning framework for identifying unknown and zero-day vulnerabilities in software supply chains.

As shown in Fig. 3.1, the methodology converts raw vulnerability and software supply chain data into structured graph representations. The proposed model then learns dependency-related patterns, updates its knowledge when new data becomes available, and produces outputs such as vulnerability labels, risk scores, and explainable indicators for analyst interpretation.

3.2 Research Design

This research adopts a quantitative experimental methodology. The quantitative aspect appears in the use of measurable variables, numerical feature matrices, graph-based node representations, and performance metrics such as accuracy, precision, recall, F1-score, AUC-ROC, false positive rate, and false negative rate. The experimental aspect appears in training and testing the proposed model and comparing its performance with baseline approaches.

The independent variables include component-level features, dependency-level features, vulnerability-related attributes, version-based indicators, temporal features, and graph-based structural features. The dependent variable is the predicted vulnerability status or risk score of a software component or dependency path. The main hypothesis is that graph-based adaptive learning can capture structural and contextual dependency patterns that improve vulnerability risk detection in software supply chains.

The study does not rely on questionnaires, interviews, or qualitative data collection. Instead, it focuses on computational experimentation using structured vulnerability and software supply chain data. The purpose is to evaluate whether a dependency-aware graph model can support detection of known vulnerable components and indicate suspicious dependency structures that may represent unknown or potential zero-day vulnerability risks.

3.3 Data Collection

Data collection is the first practical stage of the proposed methodology. It provides the raw input required for dataset construction, dependency graph generation, feature extraction, and model training. Since software supply chain risks involve vulnerabilities, packages, versions, dependencies, and project activity, the data will be collected from selected vulnerability databases, package registries, dependency files, SBOM records, and open-source repository metadata.

To keep the experimental scope feasible for a master’s thesis, the primary implementation may focus on one ecosystem, such as PyPI, together with vulnerability records from NVD and OSV. Other sources, such as GitHub metadata, SBOM files, or additional package registries, can be used as supporting or optional sources depending on dataset availability and implementation scope.

Table 3.1. Data sources, collection methods, extracted fields, and intended usage.

No.	Data Source	Collection Method	Extracted Data	Usage
1	Vulnerability records	NVD API 2.0, OSV API, GHSA advisory records, and CVE identifiers where available [1].	Vulnerability ID, affected package, affected version range, severity score, description, disclosure date, and CWE/CVE metadata.	Generate binary labels (1 = known vulnerable, 0 = non-vulnerable where applicable) and vulnerability-related features.
2	Package registry data	PyPI JSON API as the main ecosystem source; npm, Maven Central, or NuGet may be considered in extended experiments.	Package name, versions, release dates, maintainers, dependencies, and update frequency.	Build package metadata and support component-level and version-based analysis.
3	Dependency specification files	Extraction from requirements.txt, pyproject.toml, setup.py, package.json, pom.xml, or similar files depending on the ecosystem.	Direct dependencies, package requirements, version constraints, and dependency versions.	Construct direct dependency edges and derive transitive dependency paths.

4	SBOM records	Existing or generated SBOM documents using standard SBOM formats when available.	Component names, component versions, licenses, suppliers, and dependency relationships.	Validate component inventory and support graph construction.
5	Repository metadata	GitHub REST API, GitLab metadata, or repository mining tools.	Commits, issues, releases, contributors, repository age, and maintenance indicators.	Capture project activity, update behavior, and temporal features.

The collected records will be organized into a unified dataset before subsequent processing. Vulnerability records will support known vulnerability labeling; package registry data will describe software components; dependency files and SBOM records will provide graph edges; and repository metadata will contribute contextual and temporal indicators.

3.4 Data Preprocessing

Data preprocessing transforms the collected records into a consistent format suitable for feature engineering, graph construction, and model training. This stage is required because vulnerability databases, package registries, dependency files, SBOM records, and repository metadata may contain missing values, duplicate records, inconsistent package names, non-standard version formats, and incompatible structures.

Table 3.2. Data preprocessing procedures and expected outputs.

No.	Preprocessing Step	Procedure	Example of Error	Purpose
1	Data cleaning	Remove irrelevant, incomplete, invalid, or corrupted records.	Records without package name or vulnerability identifier.	Improve dataset quality and reduce noise.
2	Duplicate removal	Identify and remove repeated package, vulnerability, or dependency records.	Repeated CVE entry linked to the same package version.	Prevent duplicated samples and biased model training.
3	Missing value handling	Handle missing package names, versions, severity scores, release dates, or dependency fields through removal or imputation when appropriate.	Missing release date or missing severity score.	Maintain dataset consistency and reliability.
4	Package name normalization	Standardize package names, capitalization, namespaces, and ecosystem-specific identifiers.	Django, django, and PyPI:django represented differently.	Ensure correct mapping between packages, vulnerabilities, and dependencies.

5	Version parsing and normalization	Parse and normalize version strings and affected version ranges using version parsing tools such as packaging.version in Python.	1.2.3-alpha, v2.0, >=1.0,<2.0.	Support accurate vulnerability-to-version mapping.
6	Vulnerability-to-package mapping	Link vulnerability records to affected packages and versions.	CVE record lists a product name that differs from the registry package name.	Generate known vulnerability labels and vulnerability-related features.
7	Data integration	Merge vulnerability records, package metadata, dependency relationships, SBOM data, and repository metadata.	Different schemas across NVD, OSV, PyPI, and repository metadata.	Construct a unified dataset for graph construction and model training.

After preprocessing, package names and versions will be standardized to support accurate mapping between vulnerability records and software components. Dependency relationships will be prepared as graph edges, while vulnerability labels and metadata will be assigned as node-level or edge-level attributes for later model training.

3.5 Feature Engineering and Selection

Feature engineering converts the cleaned and integrated dataset into measurable variables that describe software components, dependency relationships, vulnerability history, version behavior, graph structure, and project activity. These features form the feature matrix required by the graph-based learning model.

Feature selection will be used to remove irrelevant, duplicated, or weakly related variables. This step reduces noise and improves model stability. Features will be selected based on availability, measurability, relevance to vulnerability risk, and contribution to validation performance.

Table 3.3. Feature categories, examples, and calculation methods.

No.	Feature Category	Extracted Features	Calculation / Formula Example	Purpose
1	Component-level features	Package name, version, ecosystem, license, and package age.	package_age = current_date - first_release_date	Describe each software component.
2	Dependency-level features	Number of direct dependencies, transitive dependencies, and dependency depth.	dependency_depth = longest dependency path from a package node	Capture dependency structure.

3	Vulnerability-related features	Known vulnerability label, severity score, affected versions, and vulnerability count.	known_vuln_flag = 1 if package-version matches an affected version range; otherwise 0 when labeled safe.	Represent known vulnerability indicators.
4	Version-based features	Release date, version age, update frequency, and outdated status.	update_frequency = number_of_releases / package_age_months	Capture package evolution and update behavior.
5	Repository activity features	Commits, issues, releases, contributors, and maintenance activity.	maintenance_activity = commits_last_year / repository_age_months	Quantify project activity and maintenance behavior.
6	Graph-based features	Node degree, centrality, dependency position, and neighbor risk.	neighbor_risk = average risk score of adjacent nodes.	Support graph learning and risk propagation analysis.

The extracted features will be converted into a feature matrix X , where each row represents a software component or component version and each column represents a selected feature. This matrix will be combined with dependency relationships to construct the software supply chain graph.

3.6 Graph Construction

Graph construction transforms the feature-enriched dataset into a graph-based representation. The software supply chain graph is defined as.

$$G = (V, E, X) \quad (3.1)$$

Where V is the set of software supply chain entities (e.g., packages, versions, repositories, vulnerability indicators), E is the set of edges representing relationships among entities (e.g., direct dependencies, transitive dependencies, version relationships, vulnerability propagation paths), and X is the node feature matrix

The first-order neighborhood of a node v is given by:

$$N(v) = \{ u \in V \mid (v, u) \in E \} \quad (3.2)$$

The framework further leverages K -hop neighborhoods to capture indirect dependencies, as vulnerability risk may propagate through both direct and transitive dependencies. Following graph construction, dependency relationships serve as graph edges, extracted features are assigned to corresponding nodes, and known vulnerability information provides labels for supervised learning where available. The resulting graph dataset serves as input to the proposed model architecture.

3.7 Proposed Model Architecture

The proposed model architecture is designed to learn structural and contextual patterns from software components and dependency relationships. The architecture includes a graph input layer, graph representation learning layer, neighborhood aggregation mechanism, adaptive learning module, and vulnerability prediction layer.

For implementation consistency, the experimental model will focus on one graph learning architecture, such as a Graph Convolutional Network (GCN). Other graph neural network variants, such as GraphSAGE or Graph Attention Networks, may be considered as baselines or future extensions, but the main methodology should remain centered on a clearly defined model.

$$H^{l+1} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^l W^l) \quad (3.3)$$

In Equation (3.3), H^l is the node representation matrix at layer l , $\tilde{A} = A + I$ is the adjacency matrix with self-loops, $\tilde{D}$ is the diagonal degree matrix of $\tilde{A}$, W^l is the trainable weight matrix at layer l , and σ is an activation function such as ReLU. This layer propagation equation enables each node to update its representation using information from neighboring dependency nodes.

$$\theta^{t+1} = \alpha \cdot \theta^t + (1 - \alpha) \cdot \nabla L^t \quad (3.4)$$

Equation (3.4) represents the adaptive update mechanism, where θ_t denotes the model parameters at time t , α is a smoothing factor, and ∇L^t is the gradient of the loss function computed from newly available training data. This mechanism allows the model to update detection knowledge when new packages, versions, dependency relationships, or vulnerability records become available.

Table 3.4. Proposed model components and mathematical representation.

No.	Model Component	Function	Mathematical Representation	Output
1	Graph input layer	Receives the software supply chain graph.	$G = (V, E, X)$, Equation (3.1).	Graph structure and node features.
2	Neighborhood definition	Identifies dependency neighbors for each node.	$N(v)$, Equation (3.2).	First-order and K-hop dependency context.
3	Graph representation learning layer	Learns structural and contextual embeddings from nodes and edges.	GCN propagation, Equation (3.3).	Node embeddings.
4	Adaptive learning module	Updates detection knowledge using new packages, versions, dependencies, or vulnerability records.	Adaptive update, Equation (3.4).	Updated risk patterns.
5	Vulnerability prediction layer	Classifies components or dependency paths according to risk.	$\hat{y}_i = \text{sigmoid}(z_i)$.	Vulnerability label or risk score.

The expected output of this stage is a graph-based adaptive model that learns from both node features and dependency relationships. The learned representations support identification of known vulnerable components and suspicious dependency structures that may indicate unknown or potential zero-day vulnerability risks.

3.8 Training and Validation Strategy

Training and validation will be conducted after constructing the graph dataset and implementing the proposed architecture. The dataset will be divided into training, validation, and testing subsets. The training subset will be used to learn model parameters, the validation subset will be used for hyperparameter tuning and overfitting control, and the testing subset will be reserved for final evaluation on unseen data.

During training, the model will learn vulnerability-related patterns from node features, dependency edges, and known vulnerability labels. Since vulnerability datasets are often imbalanced, the loss function will include a class weight to reduce bias toward the majority class.

$$L = -1/N \sum_{i=1}^N [w \cdot y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (3.5)$$

In Equation (3.5), N is the number of training samples, y_i is the true label, $\hat{y}_i$ is the predicted probability, and w is the class weight used to address class imbalance. A possible setting is $w = N_{\text{non-vuln}} / N_{\text{vuln}}$, where $N_{\text{non-vuln}}$ and N_{vuln} denote the numbers of non-vulnerable and vulnerable samples, respectively.

Hyperparameters such as learning rate, number of graph layers, hidden dimensions, dropout rate, batch size, and number of training epochs will be tuned using the validation set. The final model will then be evaluated on the testing subset to assess generalization performance on unseen components and dependency structures.

3.9 Evaluation Metrics

Model evaluation measures the proposed framework's effectiveness in detecting known vulnerable components, distinguishing non-vulnerable components, and identifying suspicious dependency structures requiring further security analysis. Multiple metrics are employed because vulnerability detection requires balancing detection capability with analyst workload. Let TP denote true positives, TN true negatives, FP false positives, and FN false negatives. Table 3.5 summarizes the evaluation metrics used in this study.

Table 3.5. Evaluation metrics and their interpretation.

No.	Metric	Purpose
1	$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$	Measure the overall correctness of model predictions.
2	$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$	Measure how many components predicted as vulnerable are actually vulnerable.
3	$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$	Measure how many actual vulnerable components are correctly detected.
4	$\text{F1} = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}$	Provide a balanced measure between precision and recall.
	$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$	Measure how often safe components are incorrectly classified as vulnerable.
	$\text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}}$	Measure how often vulnerable components are incorrectly classified as safe.
5	$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}) d(\text{FPR})$	Evaluate discrimination ability across different decision thresholds.

The evaluation results will be used to assess detection capability, generalization ability, and practical usefulness. False positives are important because they may increase analyst workload, while false negatives are critical because they may allow vulnerable components to remain undetected.

3.10 Implementation Environment and Methodological Considerations

The proposed framework will be implemented in Python, using PyTorch and PyTorch Geometric for graph neural networks [2], NetworkX for graph construction and analysis [3], and scikit-learn for baseline models and evaluation metrics [4]. Data preprocessing relies on pandas, NumPy, and packaging.version. The methodology assumes that package names and versions can be normalized to map vulnerability records to registry data, and that dependency relationships are extractable from registry metadata, dependency files, or SBOM records. The graph is treated as fixed within a defined training time window, with adaptive updates applied when new data becomes available. Limitations include dependence on the quality and availability of public vulnerability data, package metadata, and dependency information. Historical data may not represent undisclosed zero-day attacks. Additionally, incomplete dependency files, missing SBOM records, inconsistent version formats, and imbalanced vulnerability labels may reduce model performance and evaluation reliability.

References for Chapter 3

- [1] National Institute of Standards and Technology, “National Vulnerability Database (NVD),” NIST, 2026.
[Online]. Available: <https://nvd.nist.gov/>
- [2] M. Fey and J. E. Lenssen, “Fast Graph Representation Learning with PyTorch Geometric,” arXiv:1903.02428, 2019.
- [3] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in Proceedings of the 7th Python in Science Conference, 2008, pp. 11-15.
- [4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.